

Bridge Automation Framework - Detailed Explanation

1. Framework Architecture

This project is built on the **Hybrid Framework** design, primarily utilizing the **Page Object Model (POM)** pattern.

```
graph TD
    User[Tester] -->|Run Command| Pytest[Pytest Runner]
    Pytest -->|Setup| ConfTest[confTest.py]
    ConfTest -->|Read| Config[config.ini]
    ConfTest -->|Init| Driver[Selenium WebDriver]
    subgraph "Test Layer (tests/)"
        AuthTests[tests/features/auth]
        RegTests[tests/features/registration]
        DashTests[tests/features/dashboard]
    end
    subgraph "Page Layer (pages/)"
        BasePage[BasePage (Common Methods)]
        LoginPage[LoginPage]
        RegPage[RegistrationPage]
    end
    subgraph "Utilities (utilities/)"
        Logger[custom_logger.py]
        Reader[read_properties.py]
        DataUtils[data_utils.py]
    end
    Driver --> AuthTests
    AuthTests -->|Uses| LoginPage
    LoginPage -- inherits --> BasePage
    BasePage -->|Actions| Driver
    AuthTests -->|Log| Logger
    AuthTests -->|Data| DataUtils
```

2. Folder Structure Breakdown

tests/ (Test Layer)

This is where all your test scripts live. It is organized into:

- **test/**: Contains the actual feature test logic (e.g., **auth**, **registration**, **dashboard**).
- **scripts/**: Contains standalone utility or exploration scripts.
- **test_order/**: Contains "Pipeline" scripts like **test_order.bat** to run tests in a specific sequence.
- **confTest.py**: The "heart" of the test setup.

This follows the **Page Object Model**.

- **base_page.py**: Contains generic methods like **do_click**, **do_send_keys**, **get_text**. It handles **Explicit Waits** automatically, so your tests are stable even if the internet is slow.
- ***_page.py**: Each page (Login, Registration) has its own class. This separates "Locators" (IDs, XPaths) from "Test Logic".

utilities/ (Helper Layer)

custom_logger.py:

- Generates **timestamped logs** (e.g., `automation_2024...log`).
- **New:** Creates `latest_reversed.log` so you can see the newest actions at the top.
- Prints logs to the Console in real-time.
- **read_properties.py:** Reads data from `config.ini` so you don't hardcode URLs or passwords in your scripts.
- **data_utils.py:** Generates random emails, names, and dummy files (`dummy.pdf`) for upload testing.

`config/` (Configuration)

- **config.ini:** The "Control Center". Change the URL, Browser, or Payment Card details here without touching code.

`logs/` & `DOM/` (Artifacts)

- **logs/:** Stores execution history.
- **DOM/:** Stores HTML dumps of pages if you need to debug *why* an element wasn't found (captured during specific test steps).

3. How a Test Runs (The Flow)

1. **Trigger:** You run `pytest tests/features/auth/test_login.py`.
2. **Setup:** `conftest.py` kicks in -> Reads `config.ini` -> Launches Chrome -> Maximizes Window.

Execution:

- The Test Script (`test_login.py`) creates a `LoginPage` object.
 - It calls `login_page.login_with_manual_captcha()`.
 - The `LoginPage` uses `BasePage` methods to find elements and type data.
 - Logs are written to `logs/` and the Console.
4. **Verification:** The test checks if "Dashboard" elements are visible.
 5. **Teardown:** `conftest.py` closes the browser.

6. **Post-Process:** `custom_logger` generates the `latest_reversed.log`.

4. Key Improvements (Recent)

- **Organized Tests:** Moved from a flat list to `auth/`, `registration/`, etc.
- **Smart Logging:** You now have "latest first" logs and console output.
- **Clean Workspace:** Stray `.txt` and `.html` files are now automatically sorted into `pytest log/` and `DOM/`.